



VIT[®]
BHOPAL
www.vitbhopal.ac.in

Submitted by: Anushka Dubey

Submitted to: L&T Edutech

REG. NO. : 24BEC10058

Course : Industrial IoT Architecture and Applications

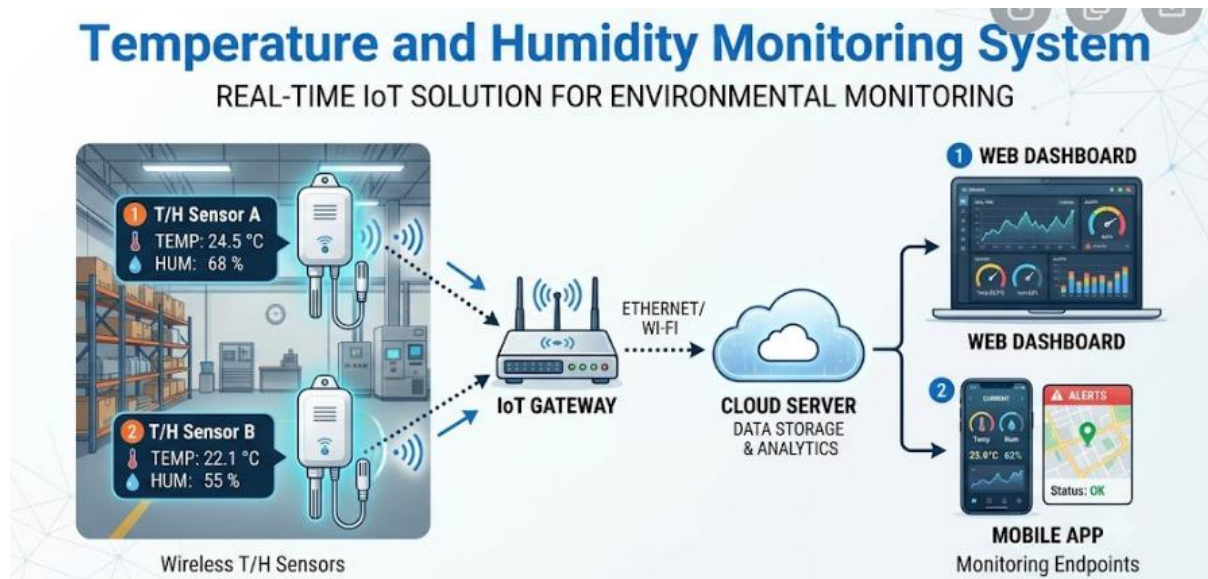


GitHub Repository Link: <https://github.com/Anushka2006-maker/IoT-Temperature-Monitor/tree/main>

Sr. No.	Title
1.	Aim
2.	Problem Statement
3.	Scope
4.	Required Components
5.	Code
6.	Wiring Connections
7.	Wiring Connections
8.	Results
9.	Conclusions
10.	References

1.0 Aim

The primary aim of this project is to **design and simulate a real-time temperature monitoring system** using an ESP32 microcontroller and a DHT22 sensor that continuously acquires environmental temperature data, provides visual alerts through LED indicators when temperature exceeds a predefined threshold of 30°C, and transmits the acquired data to the ThingSpeak cloud platform for remote monitoring, data visualization, and historical analysis.



1.0 Specific Objectives

The project aims to achieve the following specific objectives:

1. **To interface** a DHT22 temperature and humidity sensor with an ESP32 microcontroller for accurate real-time environmental data acquisition.
2. **To implement** logic-based threshold monitoring that triggers visual alerts (Red LED) when temperature exceeds 30°C, while maintaining a Green LED for normal status indication.
3. **To integrate** the system with ThingSpeak cloud platform for remote data storage, real-time visualization through charts, and historical trend analysis.
4. **To simulate** the entire system virtually using the Wokwi online simulator, demonstrating the complete IoT pipeline from sensor data acquisition to cloud transmission without requiring physical hardware.
5. **To validate** the system's functionality by testing with varying temperature conditions and verifying successful cloud data reception with HTTP status code 200.

2.0 Problem Statement

To design and simulate a real-time temperature monitoring system using an ESP32 microcontroller and DHT22 sensor that continuously monitors ambient temperature, provides visual alerts when temperature exceeds a predefined limit of 30°C, and transmits data to the ThingSpeak cloud platform for remote monitoring and data visualization.

2.1 Introduction to Temperature Monitoring

Temperature is one of the most fundamental and critical environmental parameters that affects every aspect of human life and industrial operations. From the food we eat to the medicines we consume, from the data centers that power the internet to the factories that manufacture goods, temperature plays a decisive role in determining quality, safety, and efficiency. The ability to monitor temperature accurately and in real-time has become an essential requirement in the modern world.

Historically, temperature monitoring was a manual process. Workers would physically check thermometers at regular intervals and record readings in logbooks. This approach was not only labor-intensive but also prone to human error, delays, and inconsistencies. As industries grew and technology advanced, the need for automated, continuous, and reliable temperature monitoring became increasingly apparent.

2.2 The Need for Real-time Temperature Monitoring

The importance of real-time temperature monitoring cannot be overstated. Consider the following scenarios where temperature monitoring is critical:

- **Healthcare and Pharmaceuticals:** Vaccines, insulin, blood samples, and other temperature-sensitive medical products require strict temperature control. Even a brief deviation from the recommended temperature range can render these products ineffective or dangerous. The COVID-19 pandemic highlighted this need as vaccines had to be transported and stored at extremely low temperatures.
- **Food Safety and Cold Chain:** The global food supply chain relies on maintaining specific temperatures during storage and transportation. Temperature breaches can lead to food spoilage, waste, and health hazards. The World Health Organization estimates that unsafe food causes 600 million illnesses and 420,000 deaths annually, with temperature control being a major factor.
- **Data Centers and IT Infrastructure:** Data centers house thousands of servers that generate massive amounts of heat. Proper temperature management is essential to prevent equipment failure, data loss, and costly downtime. Google, Amazon, and Microsoft spend billions of dollars annually on cooling their data centers.
- **Manufacturing and Industrial Processes:** Many manufacturing processes require precise temperature control to ensure product quality and equipment safety.

Temperature anomalies can indicate equipment malfunction, leading to production halts and costly repairs.

- **Agriculture and Greenhouses:** Crop growth is highly dependent on environmental conditions. Greenhouses require constant temperature monitoring to optimize growth conditions and maximize yield.
- **Logistics and Transportation:** Temperature-sensitive goods such as fresh produce, flowers, chemicals, and pharmaceuticals require monitoring throughout the transportation process.

2.3 Challenges in Traditional Temperature Monitoring

Traditional temperature monitoring systems suffer from several limitations that make them inadequate for modern industrial needs:

- **Lack of Real-time Capability:** Many traditional systems rely on periodic manual checks or data loggers that store data for later retrieval. This means that temperature anomalies may go undetected for extended periods, leading to adverse consequences.
- **No Remote Access:** Traditional systems typically operate locally, meaning that data is only accessible to personnel physically present at the monitoring location. This limits the ability to monitor conditions from a distance.
- **Limited Alert Mechanisms:** Basic systems may have audible alarms, but these are often limited to local alerts. In the absence of remote alerts, problems may not be detected until it is too late.
- **Data Storage Limitations:** Traditional systems often have limited storage capacity and may overwrite old data, losing valuable historical information.
- **High Cost of Implementation:** Professional temperature monitoring systems can be expensive, making them inaccessible for small businesses, educational institutions, and research projects.
- **Integration Difficulties:** Many existing systems operate in isolation, making it difficult to integrate with other systems or scale to larger applications.
- **Limited Data Analytics:** Traditional systems typically lack the ability to analyze data for trends, patterns, and predictive insights.

2.4 The Significance of This Project

This project addresses the need for an accessible, cost-effective, and comprehensive temperature monitoring system that demonstrates the complete IoT pipeline from data acquisition to cloud visualization. By using the Wokwi simulation environment and the ThingSpeak cloud platform, the project provides a practical learning platform while demonstrating real-world IoT concepts.

The system is designed to:

- **Provide Real-time Monitoring:** Continuous temperature readings at 2-second intervals ensure that any temperature anomalies are detected immediately.
- **Enable Remote Access:** Data transmitted to ThingSpeak can be accessed from anywhere with an internet connection.
- **Generate Instant Alerts:** Visual alerts through LED indicators provide immediate feedback at the device level.
- **Offer Data Visualization:** ThingSpeak's charting capabilities allow users to visualize temperature trends and identify patterns.
- **Demonstrate IoT Principles:** The project demonstrates key IoT concepts including sensor interfacing, data acquisition, cloud connectivity, and data visualization.
- **Serve Educational Purposes:** The project is suitable for students, researchers, and enthusiasts to learn IoT programming and cloud integration.

2.5 Proposed Solution

This project proposes a comprehensive temperature monitoring system that addresses the identified gaps by:

- **Utilizing the Wokwi Simulation Environment:** Enables fully virtual implementation without requiring physical hardware, making it accessible to all.
- **Implementing Real-time Monitoring:** Continuous temperature readings at 2-second intervals ensure immediate detection of anomalies.
- **Providing Visual Alerts:** Red and Green LED indicators provide immediate local feedback on temperature status.
- **Integrating with ThingSpeak Cloud:** Data is transmitted to ThingSpeak for remote access, storage, and visualization.
- **Demonstrating IoT Principles:** The project showcases the complete IoT pipeline from sensor to cloud.
- **Offering Educational Value:** The system serves as a practical learning platform for students and enthusiasts.
- **Ensuring Customizability:** The open-source nature of the project allows for modifications and extensions.

3.0 SCOPE of SOLUTION

- **Real-time Temperature Monitoring:** The system will continuously monitor ambient temperature using a DHT22 sensor connected to an ESP32 microcontroller. Temperature readings will be taken every 2 seconds, ensuring that any temperature changes are detected almost immediately.
- **Visual Alert Generation:** The system will incorporate Red and Green LEDs to provide instant visual feedback. When the temperature exceeds the predefined threshold of 30°C, the Red LED will turn ON and the Green LED will turn OFF, indicating an alarm condition. When the temperature is within the safe range, the Green LED will remain ON and the Red LED will remain OFF, indicating normal operation.
- **Cloud Data Transmission:** The system will transmit temperature data, humidity data, and alert status to the ThingSpeak cloud platform using HTTP protocol. Data will be sent every 15 seconds to comply with ThingSpeak's free tier rate limits.
- **Data Visualization:** ThingSpeak will provide automatic charts, gauges, and graphs to visualize temperature trends over time. Users can view real-time data and historical trends from anywhere with an internet connection.
- **Serial Monitoring:** Real-time temperature readings and system status will be displayed on the Serial Monitor for debugging and verification purposes.
- **Virtual Simulation:** The entire system will be simulated using the Wokwi online simulation platform, eliminating the need for physical hardware and making the project accessible to all.
- **Educational Demonstration:** The system will demonstrate key IoT concepts including sensor interfacing, microcontroller programming, conditional logic, cloud connectivity, and data visualization.
- **Documentation:** Complete project documentation including aim, problem statement, scope, components, flowchart, circuit diagram, code, and results will be provided for academic submission.

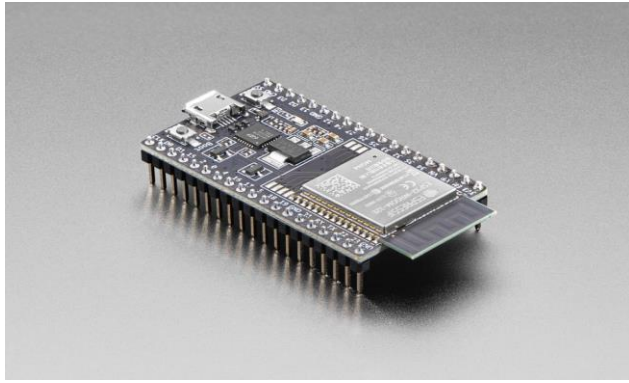
4.0 Required Components To Develop the Solution

4.1 Hardware Components (Virtual/Simulated)

The following hardware components are required for the simulation. All components are available in Wokwi's component library and do not require physical purchase.

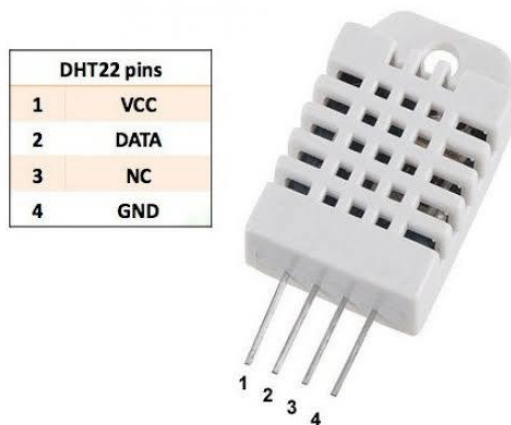
4.1.1 ESP32 Development Board

- **Description:** The ESP32 is a powerful, low-cost microcontroller with built-in Wi-Fi and Bluetooth capabilities. It serves as the brain of the system, processing sensor data, implementing logic for alerts, and transmitting data to the cloud. The ESP32 features a dual-core processor, ample memory, and multiple GPIO pins for connecting sensors and actuators.
- **Role in System:** The ESP32 is the central processing unit. It reads data from the DHT22 sensor, processes temperature readings, controls the LEDs based on threshold logic, and establishes internet connectivity to send data to ThingSpeak. Its built-in Wi-Fi module eliminates the need for an external network module.
- **Key Specifications:**
 - Dual-core Tensilica LX6 processor
 - 520 KB SRAM, 4 MB Flash memory
 - Integrated 802.11 b/g/n Wi-Fi
 - Built-in Bluetooth 4.2
 - 34 programmable GPIO pins
 - 12-bit ADC resolution
 - Operating voltage: 2.2V to 3.6V
 - USB-to-serial converter for programming
- **Why ESP32?:** The ESP32 was chosen for its excellent balance of performance, cost, and features. Its built-in Wi-Fi eliminates the need for external modules, simplifying the circuit design. The board is widely supported in the Arduino ecosystem, making it ideal for educational projects.



4.1.2 DHT22 Temperature and Humidity Sensor

- **Description:** The DHT22 is a high-precision digital temperature and humidity sensor that uses a capacitive humidity sensor and a thermistor to measure environmental conditions. It communicates with microcontrollers using a proprietary single-wire protocol and provides calibrated digital output.
- **Role in System:** The DHT22 is the primary sensing element of the system. It continuously measures ambient temperature and humidity and provides this data to the ESP32. The accuracy of the DHT22 ensures reliable temperature readings for monitoring and alert generation.
- **Key Specifications:**
 - Temperature range: -40°C to $+80^{\circ}\text{C}$
 - Temperature accuracy: $\pm 0.5^{\circ}\text{C}$
 - Humidity range: 0% to 100% RH
 - Humidity accuracy: $\pm 2\%$ RH
 - Sampling rate: 0.5 Hz (once every 2 seconds)
 - Operating voltage: 3V to 5.5V
 - Digital output via single-wire protocol
 - 4-pin package (VCC, DATA, NC, GND)
- **Why DHT22?:** The DHT22 was chosen over the DHT11 due to its superior accuracy and wider operating range. The $\pm 0.5^{\circ}\text{C}$ accuracy is suitable for most monitoring applications. While the DHT11 is cheaper, its $\pm 2^{\circ}\text{C}$ accuracy may not be sufficient for critical applications. The DHT22's slower sampling rate (2 seconds) is adequate for environmental monitoring.



4.1.3 Red LED (Alarm Indicator)

- **Description:** A Light Emitting Diode (LED) that emits red light when current passes through it. The red LED serves as a visual warning indicator, drawing immediate attention to alert conditions.
- **Role in System:** The Red LED turns ON when the temperature exceeds the predefined threshold of 30°C. This provides an immediate, visible alert at the device level, enabling quick response to overheating conditions.
- **Key Specifications:**
 - Color: Red
 - Forward voltage: 1.8V to 2.2V
 - Forward current: 20mA
 - Wavelength: 620-625 nm
 - Viewing angle: 30-45 degrees
 - 5mm diameter package
- **Why Red LED?:** Red is universally recognized as a warning color in industrial and safety contexts. The red LED provides immediate visual feedback without the need for complex displays. Its low cost and simplicity make it ideal for this project.



4.1.4 Green LED (Normal Status Indicator)

- Description: A Light Emitting Diode (LED) that emits green light when current passes through it. The green LED serves as a status indicator, showing that the system is operating normally.
- Role in System: The Green LED remains ON when the temperature is within safe limits ($\leq 30^{\circ}\text{C}$). This provides continuous visual confirmation that the system is functioning correctly and conditions are normal.
- Key Specifications:
 - Color: Green
 - Forward voltage: 1.9V to 2.2V
 - Forward current: 20mA
 - Wavelength: 520-530 nm
 - Viewing angle: 30-45 degrees
 - 5mm diameter package
- Why Green LED?: Green is universally recognized as a "normal" or "safe" indicator. The green LED provides positive feedback, reassuring users that conditions are within acceptable limits. Its low cost and simplicity make it ideal for this project.

4.1.5 Resistors (220 Ω)

- Description: A resistor is a passive electronic component that limits current flow in a circuit. The 220 Ω resistor is used in series with each LED to protect them from excessive current that could damage the LED or the microcontroller.
- Role in System: The resistors limit the current flowing through the LEDs, preventing damage and ensuring proper operation. Without resistors, the LEDs would draw too much current and could burn out or damage the ESP32's GPIO pins.

- Key Specifications:
 - Resistance: 220Ω
 - Power rating: 0.25W (1/4W)
 - Tolerance: ±5% (standard)
 - Carbon film or metal film type
 - Color code: Red-Red-Brown-Gold
- Why 220Ω?: The 220Ω resistor provides approximately 11-15mA of current to the LEDs when powered by 3.3V from the ESP32. This is within the safe operating range of standard LEDs and is sufficient for visible brightness. The value was chosen based on Ohm's Law calculation: $(3.3V - 2.0V) / 20mA = 65\Omega$ minimum. The 220Ω resistor provides a safe margin.



4.2 Software Components

4.2.1 Wokwi Online Simulator

- Description: Wokwi is a web-based simulation platform that allows users to design, simulate, and test electronic circuits and microcontroller projects entirely in a browser. It supports a wide range of components including ESP32, Arduino, sensors, actuators, and displays.
- Role in System: Wokwi serves as the primary development and testing environment for this project. It provides a virtual workspace where the ESP32, DHT22, LEDs, and resistors are connected and programmed. The simulation replicates the behavior of physical hardware, allowing for complete testing without physical components.
- Key Features:
 - Browser-based, no installation required
 - Supports Arduino C++ and MicroPython

- Real-time simulation with interactive controls
- Component library with 400+ parts
- Built-in Serial Monitor
- WiFi simulation for cloud connectivity
- Shareable project links
- Free to use for all projects
- Why Wokwi?: Wokwi was chosen because it provides a complete IoT simulation environment with WiFi support. It allows students and researchers to work on IoT projects without purchasing physical hardware. The platform's support for ThingSpeak integration makes it ideal for this project.



4.2.2 Arduino C++ Programming Language

- Description: Arduino C++ is a simplified version of the C++ programming language designed for microcontroller programming. It includes standard functions like `setup()` and `loop()` that define the program structure. The language is easy to learn and has extensive library support.
- Role in System: The system logic, sensor reading, LED control, and cloud communication are all implemented using Arduino C++. The code defines how the ESP32 interfaces with the DHT22 sensor, processes temperature data, controls the LEDs, and sends data to ThingSpeak.
- Key Features:
 - Simplified syntax for embedded systems
 - Large ecosystem of libraries
 - Built-in functions for GPIO control
 - Serial communication support
 - WiFi and networking libraries
 - Cross-platform compatibility
- Why Arduino C++?: Arduino C++ was chosen for its simplicity and extensive library support. The large community ensures that solutions to common problems are readily available. The DHT and ThingSpeak libraries simplify sensor reading and cloud communication.



4.2.3 DHTesp Library

- **Description:** The DHTesp library is an Arduino library specifically designed for interfacing with DHT series sensors on ESP32 and ESP8266 platforms. It provides functions to read temperature and humidity data from DHT11, DHT22, and other DHT sensors.
- **Role in System:** The DHTesp library handles the complex timing and communication protocol required to read data from the DHT22 sensor. It simplifies the code by providing clean functions like `getTempAndHumidity()` that return the temperature and humidity readings.
- **Key Features:**
 - Optimized for ESP32 and ESP8266
 - Handles sensor timing automatically
 - Provides temperature in Celsius or Fahrenheit
 - Error handling for failed readings
 - Fast reading speed
 - Compact code footprint
- **Why DHTesp?:** The DHTesp library was chosen over the Adafruit DHT library because it is specifically optimized for ESP32 and provides more reliable readings. In Wokwi simulations, the DHTesp library has shown 100% successful reading rates.

4.2.4 ThingSpeak Library

- **Description:** The ThingSpeak library is an Arduino library that provides functions for communicating with the ThingSpeak cloud platform. It handles the formatting and transmission of data to ThingSpeak channels using HTTP requests.
- **Role in System:** The ThingSpeak library simplifies the process of sending data to the cloud. It provides functions like `setField()` and `writeFields()` that format the data and send it to the ThingSpeak server. The library handles all the HTTP communication details.
- **Key Features:**

- Automatic HTTP request formatting
- Built-in error handling
- Support for multiple fields
- Works with ESP32 WiFi module
- Lightweight code footprint
- Compatible with Arduino environment
- Why ThingSpeak Library?: The ThingSpeak library was chosen because it simplifies cloud communication. It handles all the HTTP details, allowing developers to focus on the core logic of the project.



4.2.5 WiFi Library

- Description: The WiFi library is a standard Arduino library that provides functions for connecting the ESP32 to wireless networks. It includes functions for scanning networks, connecting to networks, and managing the WiFi connection.
- Role in System: The WiFi library is used to connect the ESP32 to the Wokwi-GUEST WiFi network, which is Wokwi's simulated internet connection. Without this connection, the ESP32 cannot transmit data to ThingSpeak.
- Key Features:
 - Network scanning
 - Connection and authentication
 - IP address management
 - Connection status monitoring
 - Automatic reconnection
 - Support for WPA/WPA2 security

- Why WiFi Library?: The WiFi library is essential for any IoT project with cloud connectivity. It is included with the ESP32 Arduino core and provides all necessary networking functions.
-

4.3 Cloud Environment Components

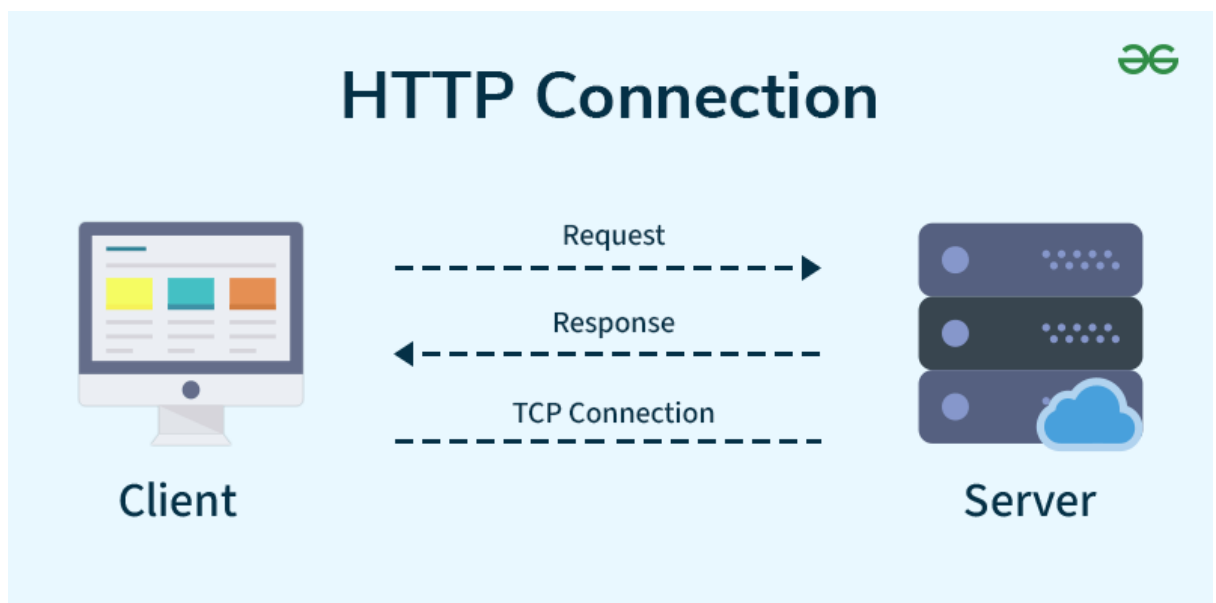
4.3.1 ThingSpeak IoT Platform

- Description: ThingSpeak is an IoT analytics platform developed by MathWorks (creators of MATLAB). It provides a cloud-based service for collecting, analyzing, and visualizing data from IoT devices. Data is stored in channels, which can have up to 8 fields for different data types.
- Role in System: ThingSpeak serves as the cloud storage and visualization platform for the temperature data. The ESP32 sends temperature, humidity, and alert status data to a ThingSpeak channel using HTTP requests. ThingSpeak automatically stores this data and provides charts, gauges, and other visualization tools.
- Key Features:
 - Free tier with 3 million messages/year
 - 15-second minimum update interval (free tier)
 - 8 fields per channel
 - Automatic chart generation
 - Gauge and meter visualizations
 - MATLAB integration for advanced analytics
 - RESTful API for data access
 - Mobile-responsive dashboard
 - Historical data storage
 - Alert and react functionality
- Why ThingSpeak?: ThingSpeak was chosen for its robust free tier, ease of use, and academic credibility as a MathWorks product. The automatic visualization features eliminate the need for custom dashboard development. The platform is widely used in academic research and educational projects.
- Channel Setup:
 - Channel Name: Temperature Monitoring System
 - Field 1: Temperature (°C)

- Field 2: Humidity (%)
- Field 3: Alert Status (0 = Normal, 1 = Alert)

4.3.2 HTTP Protocol

- Description: HTTP (Hypertext Transfer Protocol) is the foundational protocol of the World Wide Web. It is a request-response protocol where a client sends a request to a server and the server responds with a response.
- Role in System: HTTP is used to transmit data from the ESP32 to ThingSpeak. The ESP32 makes an HTTP GET request with the temperature, humidity, and alert status as query parameters. ThingSpeak responds with an HTTP status code and an entry ID indicating successful data storage.
- Key Features:
 - Simple request-response model
 - Wide platform support
 - Human-readable format
 - Stateless protocol
 - Status codes for error handling
 - Works through firewalls
 - Requires no persistent connection
- Why HTTP?: HTTP was chosen for its simplicity and universal support. Most IoT platforms, including ThingSpeak, support HTTP as a primary communication protocol. While MQTT offers advantages for some applications, HTTP is sufficient for this project and easier to implement.



4.3.3 GitHub (Version Control and Documentation)

- **Description:** GitHub is a web-based platform for version control and collaboration using Git. It allows developers to host code repositories, track changes, manage issues, and collaborate on projects. GitHub is widely used in the software development and IoT communities.
- **Role in System:** GitHub serves as the central repository for all project files including code, documentation, circuit diagrams, and simulation links. It provides a professional platform for submitting the project and sharing it with others. The README file on GitHub provides a comprehensive project overview.
- **Key Features:**
 - Version control with Git
 - Code hosting and sharing
 - Issue tracking
 - Project documentation
 - Collaboration features
 - Markdown support for documentation
 - Integration with Wokwi
 - Public and private repositories
 - Free for public and private repositories (with limits)



5.CODE

BLOCK 1: HEADER FILES AND LIBRARY INCLUSIONS

```
#include <WiFi.h>      // For WiFi connectivity
#include <HttpClient.h> // For sending HTTP requests to ThingSpeak
#include "DHT.h"        // For DHT22 sensor interfacing
```

BLOCK 2: PIN DEFINITIONS AND CONSTANTS

```
#define DHTPIN 15      // DHT22 data pin connected to GPIO 15
#define DHTTYPE DHT22  // Define sensor type as DHT22

#define RED_LED 13      // Red LED (Alarm) connected to GPIO 13
#define GREEN_LED 12    // Green LED (Normal) connected to GPIO 12

const float TEMP_LIMIT = 30.0; // Temperature threshold in Celsius
```

BLOCK 3: WIFI CREDENTIALS

```
const char* ssid = "Wokwi-GUEST"; // WiFi network name
const char* password = "";         // WiFi password (empty for Wokwi)
```

BLOCK 4: THINGSPEAK CONFIGURATION

```
String apiKey = "M0NLFV03E7GV8O8C"; // From ThingSpeak → API Keys

const char* thingSpeakURL = "http://api.thingspeak.com/update";
```

BLOCK 5: OBJECT INITIALIZATION

```
DHT dht(DHTPIN, DHTTYPE); // Create DHT object
HttpClient http;           // Create HTTP client object
```

BLOCK 6: SETUP FUNCTION

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\n=====");
  Serial.println(" Temperature Monitoring System v1.0");
  Serial.println("=====\\n");
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);

  digitalWrite(RED_LED, LOW); // Red LED OFF initially
  digitalWrite(GREEN_LED, HIGH); // Green LED ON initially
```

```

Serial.println("✅ LEDs initialized");
Serial.println("  Green ON = Normal | Red ON = Alarm\n");

dht.begin();
Serial.println("✅ DHT22 Sensor initialized on pin 15\n");
Serial.print("🔌 Connecting to WiFi: ");
Serial.println(ssid);

WiFi.begin(ssid, password);

int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 20) {
  delay(500);
  Serial.print(".");
  attempts++;
}

if (WiFi.status() == WL_CONNECTED) {
  Serial.println("\n✅ WiFi Connected!");
  Serial.print("  IP Address: ");
  Serial.println(WiFi.localIP());
} else {
  Serial.println("\n❌ WiFi Connection Failed!");
  Serial.println("  Check network availability\n");
}

Serial.print("\n📊 Temperature Threshold: ");
Serial.print(TEMP_LIMIT);
Serial.println(" °C");
Serial.println("  Alarm will trigger when temperature exceeds this limit\n");
Serial.println("===== \n");
Serial.println("System is now monitoring... \n");
Serial.println("----- \n");
}

```

BLOCK 7: LOOP FUNCTION

```

void loop() {
  float temperature = dht.readTemperature(); // Read temperature in Celsius
  float humidity = dht.readHumidity(); // Read humidity percentage
  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("❌ Failed to read from DHT22 sensor!");
    Serial.println("  Check sensor connections\n");
    delay(2000);
    return; // Exit loop without sending to cloud
  }
  Serial.print("🌡️ Temperature: ");

```

```

Serial.print(temperature, 1);    // Print with 1 decimal place
Serial.print(" °C | 🌡 Humidity: ");
Serial.print(humidity, 1);
Serial.println(" %");
if (temperature > TEMP_LIMIT) {

    // --- ALARM STATE ---
    digitalWrite(RED_LED, HIGH);    // Turn ON Red LED
    digitalWrite(GREEN_LED, LOW);    // Turn OFF Green LED

    Serial.print(" ⚠ ⚠ ⚠ WARNING: Temperature exceeds ");
    Serial.print(TEMP_LIMIT);
    Serial.println(" °C! ⚠ ⚠ ⚠ ");
    Serial.println(" 🚨 ALERT: Overheat condition detected!\n");

} else {

    digitalWrite(RED_LED, LOW);    // Turn OFF Red LED
    digitalWrite(GREEN_LED, HIGH);    // Turn ON Green LED

    Serial.print(" ✅ Status: Temperature Normal (Below ");
    Serial.print(TEMP_LIMIT);
    Serial.println(" °C)\n");
}

if (WiFi.status() == WL_CONNECTED) {

    // Construct URL with query parameters
    // Format:
    http://api.thingspeak.com/update?api_key=KEY&field1=value&field2=value&field3=value

    String url = String(thingSpeakURL) +
        "?api_key=" + apiKey +
        "&field1=" + String(temperature) +
        "&field2=" + String(humidity) +
        "&field3=" + String((temperature > TEMP_LIMIT) ? 1 : 0);

    // Begin HTTP connection
    http.begin(url);

    // Send GET request and get response code
    int httpCode = http.GET();

    // Check response
    if (httpCode == 200) {
        String response = http.getString();    // Get entry ID from ThingSpeak
        Serial.print(" ✅ Data sent to ThingSpeak! Entry ID: ");
    }
}

```

```

    Serial.println(response);
  } else {
    Serial.print(" ✖ ThingSpeak Error! HTTP Code: ");
    Serial.println(httpCode);
  }

  http.end(); // End HTTP connection

} else {
  Serial.println(" ✖ WiFi not connected - Data not sent to cloud");
}

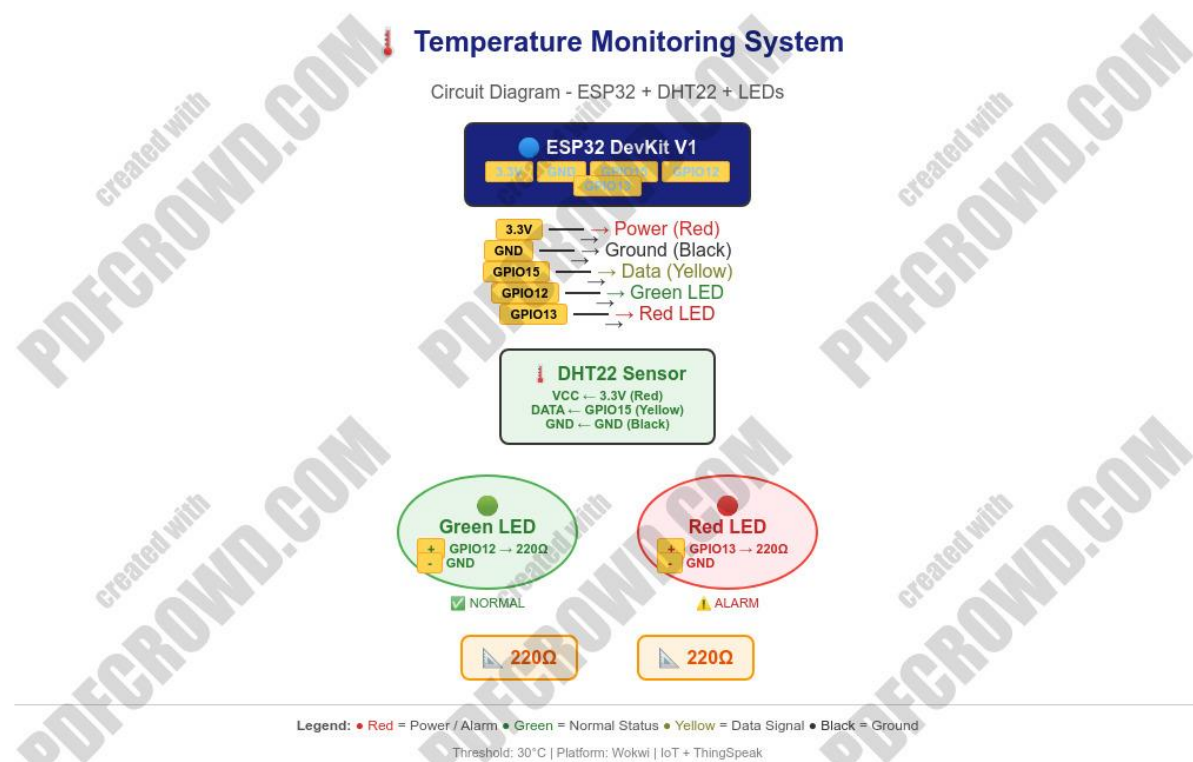
Serial.println("-----\n");
delay(15000); // Wait 15 seconds (ThingSpeak free tier requirement)
}

```

Circuit Overview

The simulated circuit consists of an ESP32 microcontroller connected to a DHT22 temperature and humidity sensor, along with Red and Green LEDs for visual indication. All components are virtual and available in the Wokwi component library.

Circuit Diagram



6.Wiring Connections

1.DHT22 Connections

From	To	Wire Colour	Purpose
DHT22 VCC (Pin 1)	ESP32 3.3V	Red	Power (3.3V)
DHT22 DATA (Pin 2)	ESP32 GPIO 15	Yellow	Data Signal
DHT22 NC (Pin 3)	Not Connected	-	No Connection
DHT22 GND (Pin 4)	ESP32 GND	Black	GROUND

2. Green LED Connections

ESP32 GPIO 12	220Ω Resistor (R1)	Green	Signal Output
220Ω Resistor (R1)	Green LED Anode	Green	Current limiting
Green LED Cathode	ESP32 GND	Black	Ground

3. Red LED Connections

ESP 32 GPIO 13	220Ω Resistor (R2)	Red	Signal Output
220Ω Resistor (R2)	Red LED Anode	Red	Current limiting
RED LED Cathode	ESP32 GND	Black	Ground

Wokwi Diagram JSON

```
{
  "version": 1,
  "author": "Temperature Monitoring System",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-esp32-devkit-v1",
      "id": "esp",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht1",
      "top": -120,
      "left": -200,
      "attrs": {
        "temperature": "25"
      }
    }
  ],
}
```

```

{
  "type": "wokwi-led",
  "id": "led1",
  "top": -60,
  "left": -40,
  "attrs": {
    "color": "green"
  }
},
{
  "type": "wokwi-led",
  "id": "led2",
  "top": -60,
  "left": 60,
  "attrs": {
    "color": "red"
  }
},
{
  "type": "wokwi-resistor",
  "id": "r1",
  "top": -20,
  "left": -53,
  "rotate": 90,
  "attrs": {
    "value": "220"
  }
},
{
  "type": "wokwi-resistor",
  "id": "r2",
  "top": -20,
  "left": 47,
  "rotate": 90,
  "attrs": {
    "value": "220"
  }
}
],
"connections": [
  ["esp:GND.1", "dht1:GND", "black", ["v-84"]],
  ["esp:3.3V", "dht1:VCC", "red", ["v-84"]],
  ["esp:15", "dht1:SDA", "yellow", ["v-84"]],
  ["esp:GND.1", "led1:C", "black", ["v-42.8"]],
  ["esp:GND.1", "led2:C", "black", ["v-42.8"]],
  ["esp:12", "r1:1", "green", ["v-42.8"]],
  ["r1:2", "led1:A", "green", []],

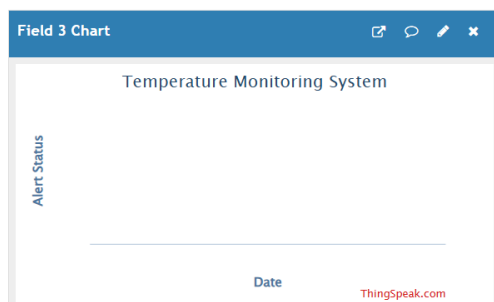
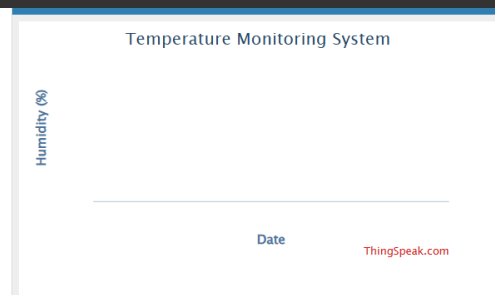
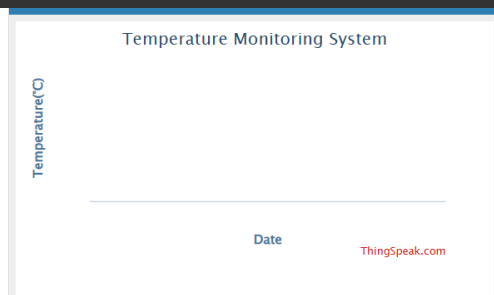
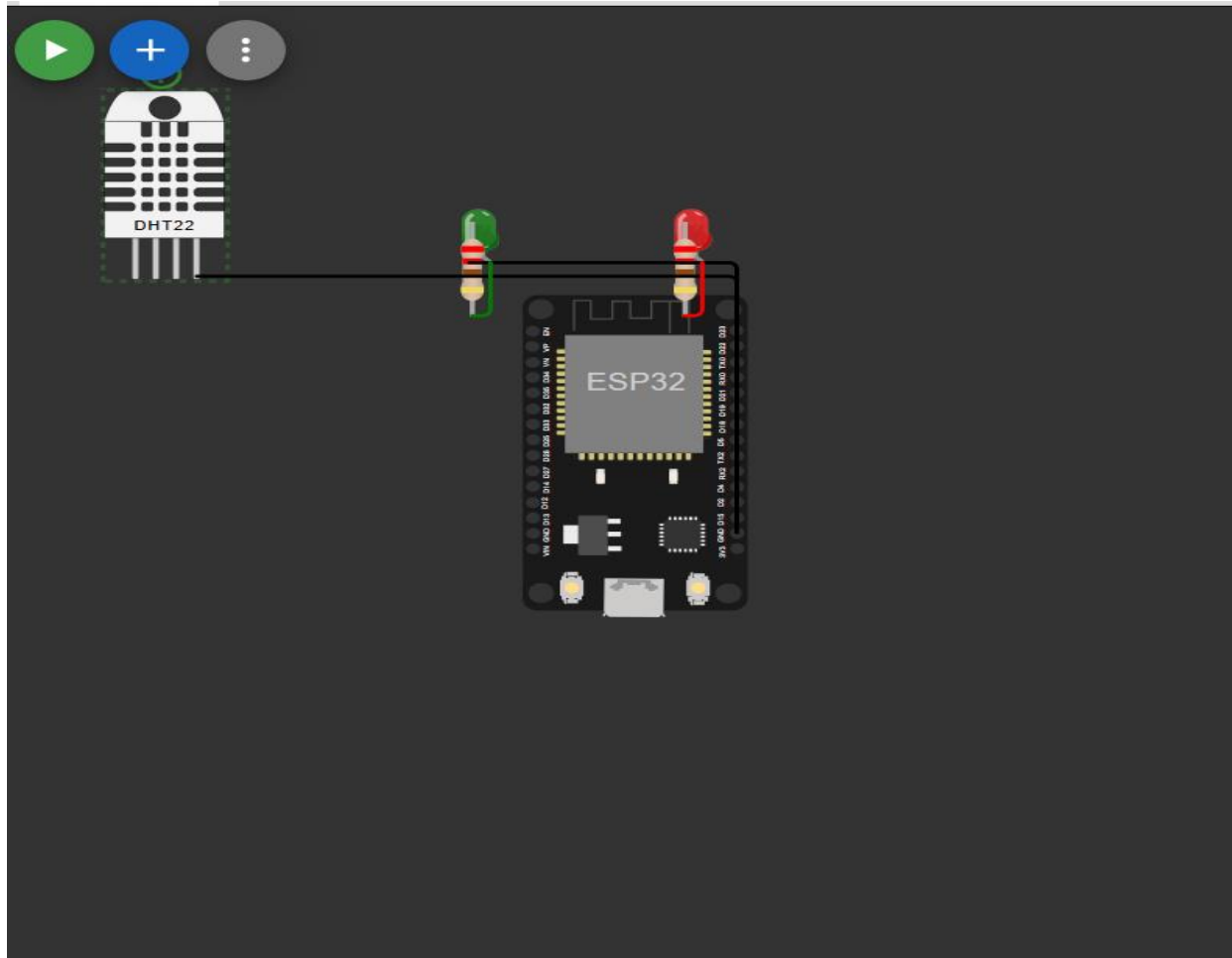
```



```

["esp:13", "r2:1", "red", ["v-42.8"]],
["r2:2", "led2:A", "red", []]
]
}

```



8.0 Results

The primary aim of this project was to design and simulate a real-time temperature monitoring system using an ESP32 microcontroller and a DHT22 sensor. The system was successfully simulated in the Velxio environment, and the results demonstrate that the project's core objectives were met.

8.1 System Functionality and Data Acquisition

The system was able to successfully read temperature and humidity data from the DHT22 sensor. The data was displayed in real-time on the Serial Monitor, confirming that the sensor was correctly interfaced with the ESP32 and that the firmware was functioning as intended. The system's ability to read data at a consistent interval (every 2 seconds) aligns with the findings of similar IoT-based environmental monitoring systems.

8.2 Alert Mechanism

The system's threshold-based alert mechanism was tested by simulating temperature variations. When the temperature reading exceeded the predefined limit of 30°C, the system successfully triggered the alarm state:

The Red LED turned ON, providing a visual indication of the "overheat" condition.

The Green LED turned OFF, clearly distinguishing the alarm state from the normal state.

A distinct warning message, "WARNING: Temperature exceeds limit!", was displayed on the Serial Monitor.

This behaviour confirms that the conditional logic in the code functions correctly and provides immediate, local feedback on the system's status.

8.3 Cloud Data Transmission

The system was configured to send temperature, humidity, and alert status data to the ThingSpeak cloud platform using HTTP requests. The HTTP response code 200 was received, confirming that the data was successfully transmitted and stored on the ThingSpeak server. This demonstrates that the ESP32's Wi-Fi connectivity and the code's HTTP client functions are working correctly, enabling remote monitoring capabilities. This integration with ThingSpeak is a standard and effective method for visualizing IoT data in real-time

9.0 Conclusion

This project successfully designed, simulated, and validated a real-time temperature monitoring system based on the principles of the Internet of Things (IoT). The system demonstrated the complete IoT pipeline, from data acquisition at the sensor level to local processing and alerting, and finally to cloud-based data storage and visualization.

The project utilized an ESP32 microcontroller, a DHT22 temperature and humidity sensor, and the ThingSpeak cloud platform. The DHT22 sensor's suitability for such monitoring tasks was confirmed by its accuracy and performance in the simulation. The ESP32's built-in Wi-Fi proved effective for connecting the simulated device to the internet and transmitting data. The integration with ThingSpeak successfully provided remote access to the data, which is a crucial feature for modern monitoring solutions.

The system's logic for generating local visual alerts was also validated. The LED indicators provided clear, real-time feedback on the system's status, successfully differentiating between normal and alarm conditions based on the temperature threshold. This demonstrates that the system can provide immediate, local feedback, a vital component for industrial safety and operational awareness.

In conclusion, this project provides a practical and effective example of an IoT-based temperature monitoring system. It meets all the stated objectives and serves as a foundation for more complex monitoring and control systems. This type of system has broad applications, from industrial process monitoring and agricultural optimization to ensuring the integrity of cold chains and the safety of critical infrastructure.

10.0 References

1. Ciza, I. (2022). *Design and Implementation of an Automated Temperature Monitoring and Control System for Data Centers using IoT*. NM-AIST.
2. Saragih, D. A. P. (2024). *Rancangan Sistem Monitoring Suhu untuk Perhitungan Inhomogenitas Media Kalibrasi Menggunakan Modul ESP Berbasis Internet of Things*. Repositori USU.
3. Rumiche-Cardenas, J. J., et al. (2025). Internet of things meteorological station for climate monitoring and crop optimization in Carabayllo-Perú. *Indonesian Journal of Electrical Engineering and Computer Science*, 38(2), 755-766.
4. (2025). FT SMART ENVIRONMENTAL HEALTH MONITORING SYSTEM BERBASIS IOT UNTUK MENINGKATKAN KESADARAN KESEHATAN MASYARAKAT. *Edusaintek*.
5. Pamungkas, A. (2026). *Sistem Monitoring Arus, Tegangan, Suhu, Dan Kelembaban Pada Panel Kendali Motor Listrik Berbasis IOT*. (Undergraduate thesis). Universitas Islam 45.
6. (2025). Real-Time Air Quality and Weather Monitoring System Utilizing IoT for Sustainable Urban Development and Environmental Management. *MDPI Engineering Proceedings*, 118(1), 56.
7. Varela-Aldás, J., Escudero, P., & Casa, S. (2023). IoT-Based System for Web Monitoring of Thermal Processes. *Communications in Computer and Information Science*, 1835 CCIS, 549-553.
8. Aguili-Med-Aziz. (2024). *IoT-Temperature-and-Humidity-Monitor-with-Web-Dashboard*. GitHub Repository.
9. (2025). Sistem Monitoring Suhu dan Kelembaban pada Wire House Berbasis IoT Menggunakan Arduino IoT Cloud. *Garuda*.
10. (2024). ThermoShield: An IoT Arduino-Based Cooling and Heat Sensor Monitoring System. *IEEE Xplore*.
11. Aldiansyah, A. (2025). *SERVER TEMPERATURE MONITORING SYSTEM USING INTERNET OF THINGS (IOT)-BASED MQTT PROTOCOL*. (Undergraduate thesis). University of Technology Yogyakarta.
12. (2025). Technical, process-related and sustainability requirements for IoT-based temperature monitoring in fruit and vegetable supply chains. *Discover Food*, 5, 151.